Exchange in a Box: A Deterministic Laboratory for Market Microstructure

Arya Somu

October 2026
Working draft, version 0.1

Abstract

We describe a reproducible limit-order-book laboratory built around a pure event transition function. The initial system implements single-instrument price-time priority, exact integer tick and lot arithmetic, deterministic replay, top-of-book state summaries, and the likelihood and compensator of an exponential Hawkes arrival process. We establish elementary invariants of the matching rules and specify a research program for synthetic market experiments. This document reports software properties only. It does not claim empirical fit to exchange data or profitable trading performance.

1 A transition system for an exchange

Let S_n denote the state after command n and c_n the next command. The engine is a function

$$F : S_n \times c_n \longrightarrow (S_{n+1}, o_n), \tag{1}$$

where o_n contains acceptance status and an ordered list of executions. State consists of bid and ask queues, a set of previously accepted IDs, and a monotone sequence counter. Prices $p \in \mathbb{Z}_{>0}$ are ticks and quantities $q \in \mathbb{Z}_{>0}$ are lots. The current command language contains submissions and cancellations. Matching executes against the best opposite quote, using the earliest accepted resting order at a tied price. Execution price is the resting order's limit price.

For a buy taker with limit p, a resting ask at a is eligible exactly when $a \leq p$. For a sell taker, a resting bid at b is eligible exactly when $b \geq p$. The matched quantity is $\min(q_{\text{taker}}, q_{\text{maker}})$, and matching repeats until the taker is exhausted or no eligible maker remains. Residual taker quantity enters its side of the book. Rejected commands leave state unchanged.

Proposition 1 (Quantity conservation for an accepted submission)**.** *For an accepted submission with initial quantity q, let $x_1, \ldots, x_k$ be its execution quantities and r its residual resting quantity, taking $r = 0$ when the order is fully executed. Then*

$$q = \sum_{i=1}^{k} x_i + r. \tag{2}$$

Proof. Each matching step subtracts the same positive amount x_i from the taker's remaining quantity as it emits in an execution. The loop terminates with the remaining quantity r, possibly zero. Telescoping these subtractions gives the claim. □

Proposition 2 (Uncrossed resting book)**.** *After any finite command sequence, if both sides are nonempty, the best bid is strictly below the best ask.*

Proof. The empty book satisfies the claim. Cancellation cannot create a crossing. On submission, an incoming order consumes every eligible opposing resting order until its quantity is zero or the best opposing price is outside its limit. Only in the latter case can a residual order rest, and its limit is then strictly outside the best opposing price. Induction over commands completes the proof. □

Pure transitions give a direct replay property: for any fixed command sequence C, folding F from the same initial state yields the same final state and outputs. Durable crash recovery will require a versioned persistent journal; it is not part of the initial implementation.

2 Order-book state as a mathematical object

Let B and A be the best bid and ask prices, with displayed quantities Q_B and Q_A aggregated across all orders at those prices. When both sides are present, define spread, imbalance, and microprice by

$$s = A - B, \tag{3}$$

$$I = \frac{Q_B - Q_A}{Q_B + Q_A}, \tag{4}$$

$$M = \frac{AQ_B + BQ_A}{Q_B + Q_A}. \tag{5}$$

Then $I \in [-1, 1]$ and $B \le M \le A$. These are state summaries, not forecasts. Predictive use would require out-of-sample evaluation with exchange-specific data, costs, queue position, and regime analysis.

3 An event-arrival model

As one candidate model for future simulated arrivals, consider the univariate exponential Hawkes intensity

$$\lambda(t) = \mu + \alpha \sum_{t_i < t} e^{-\beta(t - t_i)}, \qquad \mu > 0, \quad \alpha \ge 0, \quad \beta > 0, \quad \alpha/\beta < 1. \tag{6}$$

The final inequality is the usual stationary branching condition. For ordered event times $0 \le t_1 < \cdots < t_N \le T$, its exact log likelihood is

$$\ell(\mu, \alpha, \beta) = \sum_{i=1}^{N} \log \lambda(t_i) - \mu T - \frac{\alpha}{\beta} \sum_{i=1}^{N} \left(1 - e^{-\beta(T - t_i)}\right). \tag{7}$$

The implementation evaluates the pre-event intensity by a decaying recurrence, so likelihood evaluation is linear in N rather than quadratic. It also computes compensator increments

$$z_i = \int_{t_{i-1}}^{t_i} \lambda(u)\, du, \qquad t_0 = 0, \tag{8}$$

which support time-rescaling diagnostics. Under a correctly specified, continuous-time point-process model, these increments should behave as independent unit-exponential draws. That statement is a model check to perform, not a finding about the current system.

4 Experimental program and limits

The immediate verification target is mechanical: conservation, ordering, no crossed resting book, deterministic replay, and hand-calculated likelihood cases. The current test suite covers these properties on directed examples and a fixed-seed 2,000-command synthetic sequence. The sequence is a software stress check, not a statistical sample of markets.

The next study would compare Poisson and Hawkes arrival models on held-out simulated paths, then introduce interacting buy and sell flows, queue-dependent intensities, cancellations, and latency. We would predefine parameter ranges, random seeds, evaluation windows, and diagnostics before using any external market data. Parameter identifiability, finite-sample bias, nonstationarity, and sensitivity to timestamp resolution would be reported. A later empirical study would additionally need data rights, venue rules, transaction costs, and careful leakage controls.

5 Conclusion

A deterministic matching core and explicit stochastic model make market-mechanism experiments auditable. The present artifact is a tested software foundation and a mathematical specification. Its research claims remain deliberately narrow until calibrated experiments are run.